

# FUNCTIONAL SPECIFICATION DOCUMENT

## Metro E-Ticketing System



## E-Ticketing



A Self-Service Ticket Booking & Smart-Card Recharge Solution Built on ServiceNow.



**Recharge**



**Book Ticket**

# FUNCTIONAL SPECIFICATION DOCUMENT

## Metro E-Ticketing System

*A Self-Service Ticket Booking & Smart-Card Recharge Solution Built on ServiceNow*

FSD — Functional & Non-Functional Specification

# Table of Contents

## **1. INTRODUCTION**

- 1.1 Project Overview
- 1.2 Purpose

## **2. IDEATION PHASE**

- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

## **3. REQUIREMENT ANALYSIS**

- 3.1 Customer Journey map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

## **4. PROJECT DESIGN**

- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

## **5. PROJECT PLANNING & SCHEDULING**

- 5.1 Project Planning

## **6. FUNCTIONAL AND PERFORMANCE TESTING**

- 6.1 Performance Testing

## **7. RESULTS**

- 7.1 Output Screenshots

## **8. ADVANTAGES & DISADVANTAGES**

## **9. CONCLUSION**

## **10. FUTURE SCOPE**

## **11. APPENDIX**

- Source Code(if any)
- Dataset Link
- GitHub & Project Demo Link

# 1. INTRODUCTION

## 1.1 Project Overview

The Metro E-Ticketing System is a digital ticketing and smart-card recharge solution built entirely on the ServiceNow Now Platform, using the Service Catalog, Flow Designer, Catalog Client Scripts, Catalog UI Policies, and custom application tables. The system allows commuters to book a metro journey ticket or recharge their metro smart card through a self-service catalog item (“Book a Metro Ticket”), removing the need to visit a physical ticket counter.

Passengers interact with the solution through the ServiceNow Employee Center / Service Portal, where the catalog item captures journey details (source station, destination station, number of passengers, journey type), calculates the fare, collects the payment mode, and — on submission — generates a scannable QR code that acts as the passenger's digital ticket. A Flow Designer automation records every completed transaction into a dedicated “Metro Database” table for downstream tracking, reconciliation, and reporting.

## 1.2 Purpose

This Functional Specification Document (FSD) formally defines what the Metro E-Ticketing System does, who it is for, and how its components fit together. It translates the original problem statement and ideation output into concrete functional and non-functional requirements, describes the solution architecture actually implemented on the ServiceNow platform, and records the testing performed and results obtained. It is intended for reviewers, evaluators, and any developer who needs to maintain or extend the application.

# 2. IDEATION PHASE

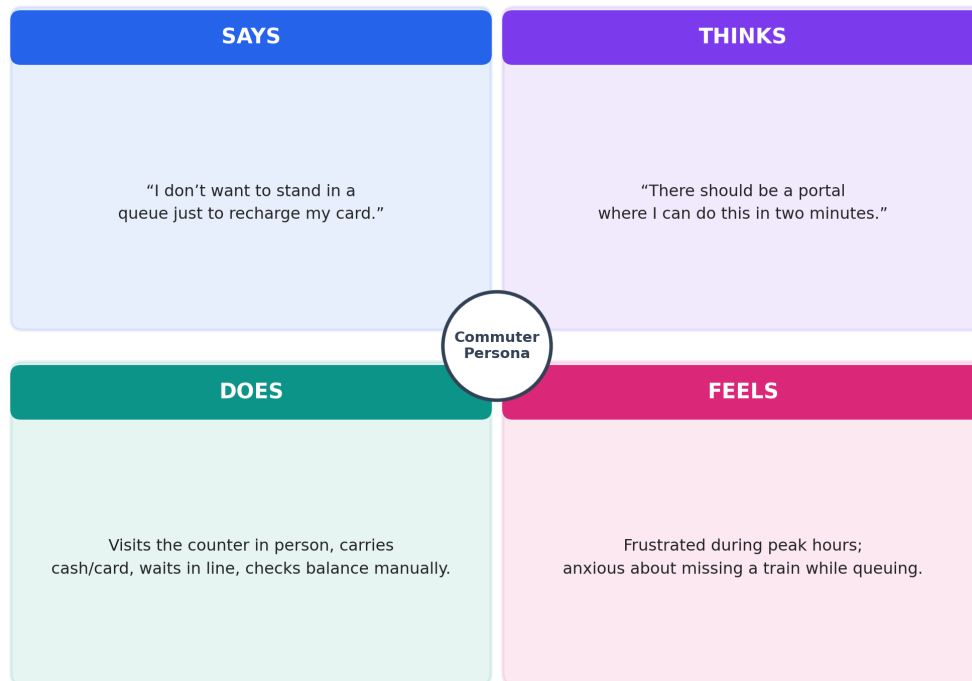
## 2.1 Problem Statement

Commuters travelling by metro rail currently depend on physical ticket counters or vending machines to purchase single-journey tickets or recharge their smart cards. This results in long queues during peak hours, dependency on cash or on-site card machines, no self-service option to book or recharge from a phone or laptop, and no single digital record a passenger can refer back to for a past journey or recharge.

*“Daily and occasional metro passengers need a fast, self-service way to book tickets and recharge smart cards, because the current counter-based process is time-consuming and inconvenient — which makes routine commuting more stressful than it needs to be.”*

## 2.2 Empathy Map Canvas

An empathy map was used to understand the commuter persona before designing the solution:



*Fig 2.2: Empathy Map Canvas for the daily metro commuter*

## 2.3 Brainstorming

Based on the empathy map, the team brainstormed and shortlisted the following ideas before converging on the final solution:

- A mobile app integrated with the metro's existing ticketing backend.
- A kiosk-based self-service terminal at each station.
- A self-service catalog item on an existing enterprise workflow platform (ServiceNow), reusable for both ticket booking and smart-card recharge.
- A QR-code-based digital ticket that can be scanned at the gate instead of a physical ticket.

The ServiceNow Service Catalog approach was selected because it allows a single request form to branch between “Recharge Metro Card” and “Book a Metro Ticket”, reuses the platform's built-in approval/tracking (Request/RITM) and portal capabilities, and lets automation (Flow Designer) persist every transaction without custom backend development.

## 3. REQUIREMENT ANALYSIS

## 3.1 Customer Journey Map

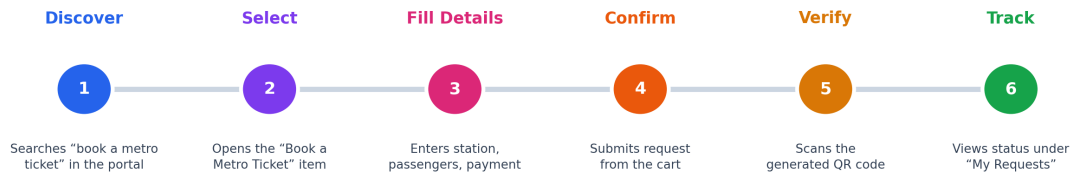


Fig 3.1: Passenger journey from discovery to request tracking

## 3.2 Solution Requirement

### Functional Requirements

| ID    | Requirement                    | Description                                                                                                                          |
|-------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| FR-1  | Service selection              | User selects either "Recharge Metro Card" or "Book a Metro Ticket" from the catalog item.                                            |
| FR-2  | Smart card details capture     | System captures smart card number and smart card name for recharge transactions.                                                     |
| FR-3  | Journey details capture        | System captures starting station, destination station, number of passengers, and journey type (Single / Return / Group).             |
| FR-4  | Automatic fare calculation     | A client script (FareCalculator) computes the fare for a single journey and for a return journey based on route and passenger count. |
| FR-5  | Input validation               | A client script validates that the number of passengers is a valid numeric value before submission.                                  |
| FR-6  | Payment mode selection         | User selects a payment mode (Credit Card, Debit Card, or NetBanking); a UI Policy shows/hides the "enter payment mode" field.        |
| FR-7  | Digital QR ticket generation   | On submission, a client script opens a modal widget that generates and displays a scannable QR code.                                 |
| FR-8  | Transaction persistence        | A Flow Designer flow triggers on the Service Catalog request and creates a record in the "Metro Database" table.                     |
| FR-9  | Request tracking               | Every submission generates a Request (REQ) and Requested Item (RITM) viewable under "My Requests".                                   |
| FR-10 | Transaction history for admins | All recharge and booking transactions are stored and viewable in the "Metro Database" table.                                         |

### Non-Functional Requirements

| ID    | Requirement     | Description                                                                                 |
|-------|-----------------|---------------------------------------------------------------------------------------------|
| NFR-1 | Usability       | Catalog form uses standard, familiar field types so passengers need no training.            |
| NFR-2 | Performance     | Fare calculation and field validation run client-side (onChange) for an instant response.   |
| NFR-3 | Reliability     | Flow Designer executes the record-creation action reliably for every submitted request.     |
| NFR-4 | Security        | Access is governed by ServiceNow's built-in roles, ACLs, and authenticated portal login.    |
| NFR-5 | Scalability     | Solution scales to more stations/fare rules/payment modes by extending variables & scripts. |
| NFR-6 | Availability    | Runs on the ServiceNow cloud instance, inheriting platform-level uptime.                    |
| NFR-7 | Maintainability | Logic is centralised in a few catalog client scripts, one UI policy, and one flow.          |

### 3.3 Data Flow Diagram

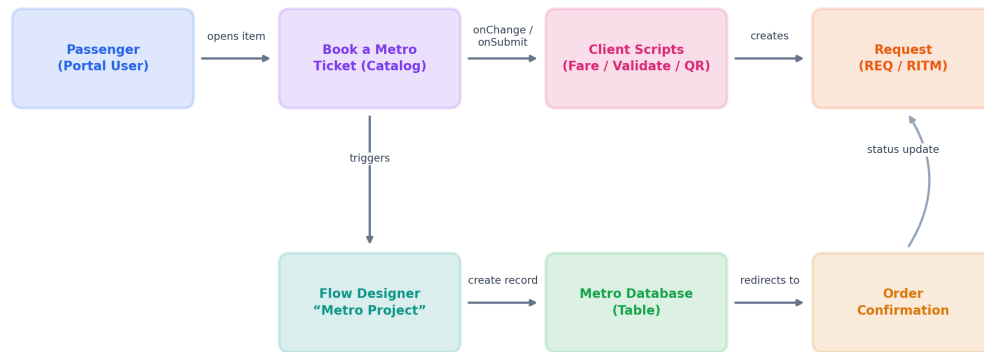


Fig 3.3: Data flow from passenger request to Metro Database record creation

### 3.4 Technology Stack

| Layer         | Technology / Component                       | Purpose                                                                   |
|---------------|----------------------------------------------|---------------------------------------------------------------------------|
| Platform      | ServiceNow Now Platform                      | Low-code platform hosting the catalog, forms, automation, and data model. |
| Presentation  | Employee Center / Service Portal             | Self-service UI where passengers search for and open the catalog item.    |
| Catalog Layer | Service Catalog Item — “Book a Metro Ticket” | Defines the request form and its 13 catalog variables.                    |
| Client Logic  | Catalog Client Scripts (JavaScript)          | FareCalculator, fieldValidation, QRGeneration.                            |

| Layer      | Technology / Component                                            | Purpose                                                             |
|------------|-------------------------------------------------------------------|---------------------------------------------------------------------|
| Form Logic | Catalog UI Policy — Field Visibility                              | Dynamically shows/hides the payment-mode field.                     |
| Automation | Flow Designer — “Metro Project” flow                              | Reads catalog variables and creates a record in the Metro Database. |
| Data Layer | Custom Tables<br>(u_metro_station_s_details,<br>u_metro_database) | Store station master data and every transaction.                    |
| Widget     | QR Ticket Widget (Service Portal widget)                          | Renders the scannable QR code inside a modal.                       |
| Tracking   | Request (REQ) / Requested Item (RITM)                             | Native ServiceNow objects used for status tracking.                 |

## 4. PROJECT DESIGN

### 4.1 Problem – Solution Fit

| Problem                                       | How the Solution Addresses It                                               |
|-----------------------------------------------|-----------------------------------------------------------------------------|
| Long queues at physical counters              | Self-service catalog item available anytime from the portal.                |
| No digital record of tickets/recharges        | Every transaction logged in the Metro Database & visible under My Requests. |
| Manual, slow fare calculation                 | FareCalculator client script computes the fare instantly.                   |
| Risk of invalid data entry                    | fieldValidation script blocks invalid values with an inline alert.          |
| No proof of purchase without a printed ticket | QRGeneration script produces a scannable QR code as the digital ticket.     |

### 4.2 Proposed Solution

The proposed solution is a single ServiceNow Service Catalog item, “Book a Metro Ticket”, that serves two purposes through one request form: (1) recharging an existing metro smart card, and (2) booking a single, return, or group journey ticket. Client-side scripts handle fare calculation, input validation, and QR-ticket generation, while a Flow Designer flow persists every completed transaction to a dedicated Metro Database table. This keeps the solution entirely within a governed, low-code platform.

## 4.3 Solution Architecture

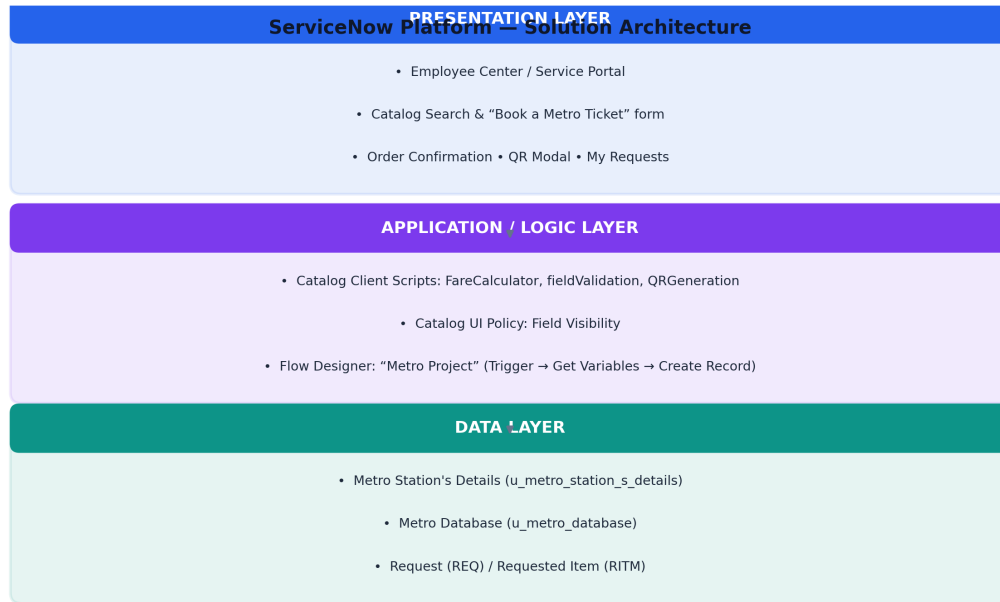


Fig 4.3: Three-tier solution architecture on the ServiceNow platform

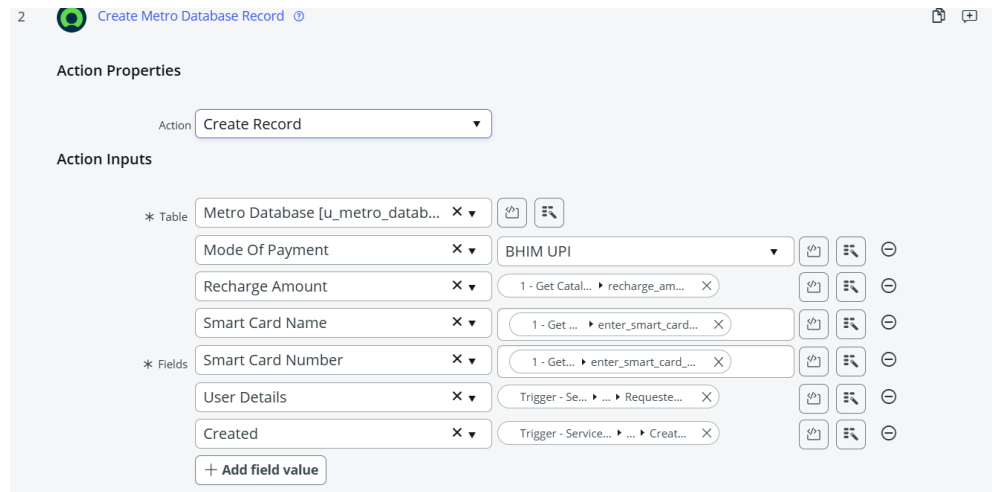


Fig 4.3(a): Flow Designer action — Create Record in the Metro Database table



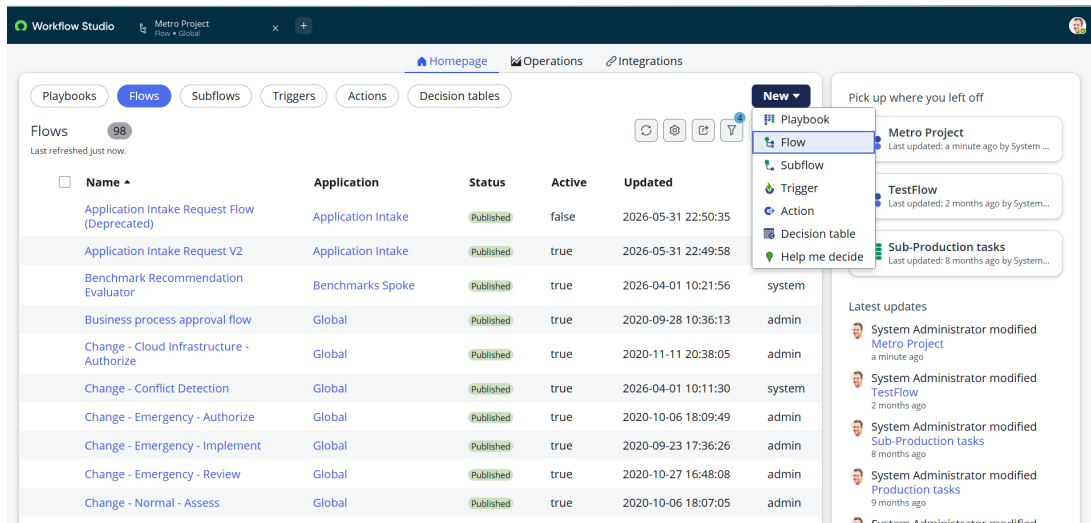


Fig 4.3(b): “Metro Project” flow listed in Flow Designer, triggered from the Service Catalog

## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

| Sprint   | Focus Area                      | Key Deliverables                                                           |  |
|----------|---------------------------------|----------------------------------------------------------------------------|--|
| Sprint 1 | Ideation & Requirement Analysis | Problem statement, empathy map, customer journey map, requirements.        |  |
| Sprint 2 | Data Model & Catalog Setup      | Custom tables, “Book a Metro Ticket” catalog item with 13 variables.       |  |
| Sprint 3 | Client-Side Logic               | FareCalculator script, fieldValidation script, Field Visibility UI Policy. |  |
| Sprint 4 | Automation & QR Ticketing       | “Metro Project” flow, QRGeneration script and QR modal widget.             |  |
| Sprint 5 | Testing & Deployment            | Functional & performance testing, portal walkthrough, documentation.       |  |

## 6. FUNCTIONAL AND PERFORMANCE TESTING

### 6.1 Performance Testing

#### Functional Test Cases

| Test ID | Scenario                                      | Expected Result                                               | Status |
|---------|-----------------------------------------------|---------------------------------------------------------------|--------|
| TC-01   | Open “Book a Metro Ticket” from portal search | Catalog item loads with valid short description & Add to Cart | Pass   |
| TC-02   | Select item & fill journey details            | Fare fields auto-populate via FareCalculator                  | Pass   |

| Test ID | Scenario                                      | Expected Result                                          | Status |
|---------|-----------------------------------------------|----------------------------------------------------------|--------|
| TC-03   | Enter non-numeric value in “No of passengers” | fieldValidation blocks the field and shows an alert      | Pass   |
| TC-04   | Select payment mode = NetBanking              | UI Policy correctly shows/hides payment mode field       | Pass   |
| TC-05   | Submit the completed request                  | QRGeneration opens a modal with a scannable QR code      | Pass   |
| TC-06   | Confirm submission after QR scan              | Order Confirmation page shown with request details       | Pass   |
| TC-07   | Check Flow Designer execution                 | New record created in Metro Database with correct values | Pass   |
| TC-08   | Open “My Requests”                            | REQ / RITM visible with correct activity history         | Pass   |

## Performance Observations

- Client scripts execute on the browser with no server round-trip, giving an instant response as the user types.
- The Flow Designer automation completes the “get variables → create record” sequence in a single trigger execution per request.
- The catalog form, confirmation screen, and QR modal all render within standard Service Portal page-load times.

Catalog UI Policy Field Visibility

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More info](#)

Applies to: A Catalog Item Application: Global

\* Catalog Item: Book a metro ticket Active: ☒

\* Short description: Field Visibility

When to Apply Script

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is Active
2. The items in the Conditions field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions: Add Filter Condition Add OR Clause

mode\_of\_payment is Others AND OR

Applies on a Catalog Item view: ☒

Applies on Catalog Tasks: ☐

Applies on Requested Items: ☐

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

On load: ☒

Reverse the effects of the catalog UI policy actions when the Conditions evaluate to false

Reverse if false: ☒

Fig 6.1(a): Catalog UI Policy — conditional field visibility configuration

The screenshot shows the 'Catalog Client Scripts' configuration page for a script named 'QR Generation'. The script is active and applies to the 'Book a metro ticket' catalog item. The script code is as follows:

```

1 function onSubmit() {
2   var work = g_form.getValue('what_do_you_want_to_do');
3
4   if (work == 2 && g_form.getValue('qr_confirmed') != 'true') {
5     var sysid = g_form.getUniqueValue();
6     var dummyURL = 'https://example.com/ticket?id=' + sysid;
7
8     sptModal.open({
9       title: "Scan Your QR Here",
10      widgets: "qr_ticket_widget",
11      widgetInput: { ticketId: dummyURL }
12    }).then(function() {
13      g_form.setValue('qr_confirmed', 'true');
14      g_form.submit();
15    });
16
17    return false;
18  }
19
20  return true;
21 }

```

On the right side, there are checkboxes for 'Applies on a Catalog Item view' (checked), 'Applies on Requested Items' (checked), and 'Applies on Catalog Tasks' (checked).

Fig 6.1(b): QRGeneration client script (onSubmit)

## 7. RESULTS

### 7.1 Output Screenshots

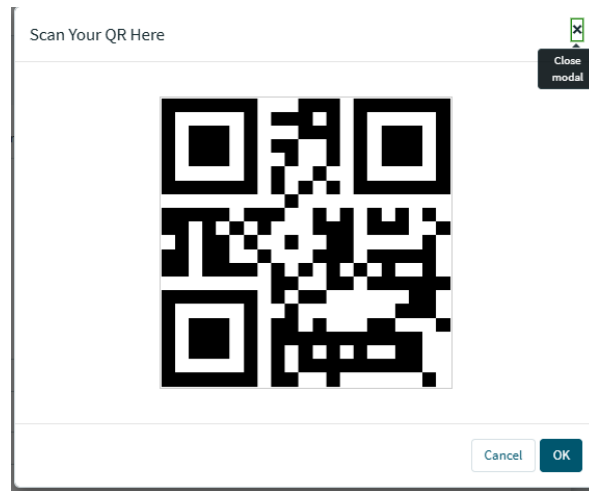
The screenshot shows the 'Book a metro ticket' catalog form. The form includes the following fields and options:

- What do you want to do?** (Indicates required)
  - ☐ Recharge Metro Card
  - ☒ Book a Metro Ticket
  - ☐ None
- Starting From**: Amserjet
- Going to**: Radburg
- No of passengers**: 1
- Type of journey**
  - ☒ Single Journey
  - ☐ Return Journey
  - ☐ Group Ticket
- Amount for single journey**: 40
- Amount including return**:
- Mode of payment**
  - ☐ Credit Card
  - ☐ Debit Card
  - ☒ Net Banking

On the right side, there is a summary box with the following information:

- Quantity**: 1
- Delivery Time**: 0 Days
- Buttons**: Add to Cart, Save as Draft, Order Now

Fig 7.1(a): "Book a Metro Ticket" catalog form — journey and payment details



*Fig 7.1(b): QR code modal generated as the digital ticket*

 An "Order Confirmation" modal window with a close button in the top right corner. It contains a "Request for" dropdown menu currently set to "System Administrator". Below this are two expandable sections: "Delivery Information (Optional)" and "Special instructions (Optional)". At the bottom right are "Cancel" and "Checkout" buttons.

*Fig 7.1(c): Order confirmation screen after ticket submission*

Home > Request Summary - REQ0010006 Search Catalog

Submitted : 2026-07-27 18:57:48  
 Request Number : **REQ0010006**  
 Estimated Delivery : 2026-07-27

| Item                                | Delivery Date | Stage | Price (each) | Quantity | Total         |
|-------------------------------------|---------------|-------|--------------|----------|---------------|
| <a href="#">Book a metro ticket</a> | 2026-07-27    |       | ---          | 1        | ---           |
|                                     |               |       |              |          | Total: \$0.00 |

*Fig 7.1(d): Request summary (REQ) generated for the submitted order*

| User Details         | Smart Card Number | Smart Card Name | Recharge Amount | Mode Of Payment | Created             |
|----------------------|-------------------|-----------------|-----------------|-----------------|---------------------|
| System Administrator | (empty)           |                 |                 | BHIM UPI        | 2026-07-26 18:24:46 |
| System Administrator | (empty)           |                 |                 | BHIM UPI        | 2026-07-26 18:50:46 |
| System Administrator | (empty)           |                 |                 | BHIM UPI        | 2026-07-27 18:57:50 |
| System Administrator | 856045            | Ritika          | 400             | BHIM UPI        | 2026-07-26 17:54:57 |
| System Administrator | (empty)           |                 |                 | BHIM UPI        | 2026-07-27 18:56:22 |

Fig 7.1(e): Metro Database table showing recorded recharge/booking transactions

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Eliminates queues at physical counters — tickets and recharges from any device with portal access.
- Instant fare calculation and input validation reduce user errors before submission.
- QR-based digital ticket removes dependency on printed tickets.
- Built entirely on a governed low-code platform (ServiceNow) — no separate backend needed.
- Every transaction is automatically logged and traceable via Request/RITM tracking.

### Disadvantages

- Solution depends on an active ServiceNow instance and user login; no offline mode.
- Current implementation demonstrates a simulated QR/payment flow rather than a live payment gateway or turnstile integration.
- Fare rules and station lists are maintained manually rather than synced from a live transport feed.
- Very high-volume peak-hour usage would need load testing against ServiceNow platform limits.

## 9. CONCLUSION

The Metro E-Ticketing System demonstrates that a complete self-service ticketing and smart-card recharge workflow can be delivered on the ServiceNow platform using the Service Catalog, Catalog Client Scripts, a UI Policy, and a Flow Designer automation, without building a separate application or database server. The solution meets the core functional requirements identified during the ideation phase — service selection, fare calculation, validation, QR ticket generation, and transaction logging — and was verified through functional testing of each catalog variable, script, and flow action. It directly addresses the original problem of counter-based queuing by giving commuters an always-available digital alternative.

## 10. FUTURE SCOPE

- Integrate a live payment gateway in place of the simulated payment-mode capture.
- Connect the QR ticket to actual station turnstile/gate hardware for real-time validation.
- Sync station master data and fares from the transport authority's live system.
- Add a mobile-optimised or native app view for on-the-go booking.
- Build a reporting/analytics dashboard on top of the Metro Database table.
- Extend the catalog item with multi-language support and accessibility improvements.

## 11. APPENDIX

### 11.1 Source Code (if any)

#### FareCalculator — Catalog Client Script (onChange, applies to Type of Journey)

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') { return; }
    var source = g_form.getValue('starting_from');
    var destination = g_form.getValue('going_to');
    var numPassengers = parseInt(g_form.getValue('no_of_passengers'));
```

#### fieldValidation — Catalog Client Script (onChange, applies to No of Passengers)

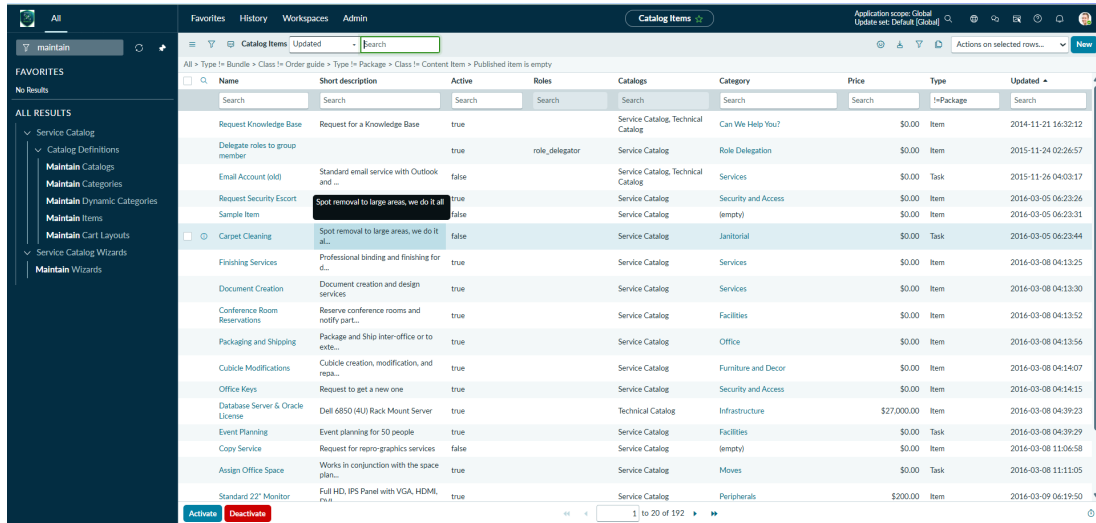
```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') { return; }
    if (isNaN(newValue)) {
        alert('Please enter the values in number format only');
    }
}
```

#### QRGeneration — Catalog Client Script (onSubmit)

```
function onSubmit() {
    var work = g_form.getValue('what_do_you_want_to_do');
    if (work == 2 && g_form.getValue('qr_confirmed') != 'true') {
        var sysId = g_form.getUniqueValue();
        var dummyURL = 'https://example.com/ticket?id=' + sysId;
        spModal.open({ title: 'Scan Your QR Here', widget: 'qr_ticket_widget',
            widgetInput: { ticketUrl: dummyURL } }).then(function() {
            g_form.setValue('qr_confirmed', 'true');
            g_form.submit();
        });
        return false;
    }
    return true; }
```

## 11.2 Dataset Link

| Table                                               | Key Fields                                                                                  |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------|
| Metro Station's Details (u_metro_station_s_details) | Customer Name, Record Number, Station Name                                                  |
| Metro Database (u_metro_database)                   | Smart Card Number, Smart Card Name, Recharge Amount, Mode of Payment, User Details, Created |



The screenshot displays the ServiceNow Catalog Item Management interface. The left sidebar shows navigation options like 'Service Catalog', 'Catalog Definitions', 'Maintain Catalogs', 'Maintain Dynamic Categories', 'Maintain Items', 'Maintain Cart Layouts', 'Service Catalog Wizards', and 'Maintain Wizards'. The main area shows a list of catalog items with columns: Name, Short description, Active, Roles, Catalogs, Category, Price, Type, and Updated. The list includes items like 'Request Knowledge Base', 'Delegate roles to group member', 'Email Account (add)', 'Request Security Escort', 'Sample Item', 'Carpet Cleaning', 'Finishing Services', 'Document Creation', 'Conference Room Reservations', 'Packaging and Shipping', 'Cubicle Modifications', 'Office Keys', 'Database Server & Oracle License', 'Event Planning', 'Copy Service', 'Assign Office Space', and 'Standard 22" Monitor'.

| Name                             | Short description                           | Active | Roles          | Catalogs                           | Category            | Price       | Type | Updated             |
|----------------------------------|---------------------------------------------|--------|----------------|------------------------------------|---------------------|-------------|------|---------------------|
| Request Knowledge Base           | Request for a Knowledge Base                | true   |                | Service Catalog, Technical Catalog | Can We Help You?    | \$0.00      | Item | 2014-11-21 16:32:12 |
| Delegate roles to group member   |                                             | true   | role_delegator | Service Catalog                    | Role Delegation     | \$0.00      | Item | 2015-11-24 02:26:57 |
| Email Account (add)              | Standard email service with Outlook and ... | false  |                | Service Catalog, Technical Catalog | Services            | \$0.00      | Task | 2015-11-26 04:03:17 |
| Request Security Escort          | Spot removal to large areas, we do it all   | true   |                | Service Catalog                    | Security and Access | \$0.00      | Item | 2016-03-05 06:23:26 |
| Sample Item                      |                                             | false  |                | Service Catalog                    | (empty)             | \$0.00      | Item | 2016-03-05 06:23:31 |
| Carpet Cleaning                  | Spot removal to large areas, we do it all   | false  |                | Service Catalog                    | Janitorial          | \$0.00      | Task | 2016-03-05 06:23:44 |
| Finishing Services               | Professional binding and finishing for d... | true   |                | Service Catalog                    | Services            | \$0.00      | Item | 2016-03-08 04:13:25 |
| Document Creation                | Document creation and design services       | true   |                | Service Catalog                    | Services            | \$0.00      | Item | 2016-03-08 04:13:30 |
| Conference Room Reservations     | Reserve conference rooms and notify part... | true   |                | Service Catalog                    | Facilities          | \$0.00      | Item | 2016-03-08 04:13:52 |
| Packaging and Shipping           | Package and Ship inter-office or to exte... | true   |                | Service Catalog                    | Office              | \$0.00      | Item | 2016-03-08 04:13:56 |
| Cubicle Modifications            | Cubicle creation, modification, and repa... | true   |                | Service Catalog                    | Furniture and Decor | \$0.00      | Item | 2016-03-08 04:14:07 |
| Office Keys                      | Request to get a new one                    | true   |                | Service Catalog                    | Security and Access | \$0.00      | Item | 2016-03-08 04:14:15 |
| Database Server & Oracle License | Dell 6850 (4U) Rack Mount Server            | true   |                | Technical Catalog                  | Infrastructure      | \$27,000.00 | Item | 2016-03-08 04:39:23 |
| Event Planning                   | Event planning for 50 people                | true   |                | Service Catalog                    | Facilities          | \$0.00      | Task | 2016-03-08 04:39:29 |
| Copy Service                     | Request for repro-graphics services         | false  |                | Service Catalog                    | (empty)             | \$0.00      | Item | 2016-03-08 11:06:58 |
| Assign Office Space              | Works in conjunction with the space plan... | true   |                | Service Catalog                    | Moves               | \$0.00      | Task | 2016-03-08 11:11:05 |
| Standard 22" Monitor             | Full HD, IPS Panel with VGA, HDMI, and ...  | true   |                | Service Catalog                    | Peripherals         | \$200.00    | Item | 2016-03-09 06:19:50 |

Fig 11.2: ServiceNow Catalog Item Management view listing configured catalog items

## 11.3 GitHub & Project Demo Link

GitHub Repository: <https://github.com/rithika1056-collab/Metro-Ticket-Generating-System-in-ServiceNow.git>

Project Demo Link: [https://drive.google.com/file/d/1MsJlKS\\_Yg3icTJiGzOrjaqkTiavP2xDJ/view?usp=sharing](https://drive.google.com/file/d/1MsJlKS_Yg3icTJiGzOrjaqkTiavP2xDJ/view?usp=sharing)